

Convolutional Neural Networks (CNN)

Previous session

In the previous session, you learned:

➤ How **convolutions work for RGB images with three channels.**

➤ **ReLU activation layer, Pooling layer, Flatten layer**

Introducing non-linearity

(Max, Average, and Min) in reducing spatial dimensions while retaining key features.

Bridge role between the convolutional (feature extraction) part and the fully connected (classification) part of the network.

Today's Session Overview

In today's session, you will:

Learn key **regularization technique** to improve generalization

- Batch Normalization in detail

Batch Normalization Implementation in MLP

```
from keras.models import Sequential
```

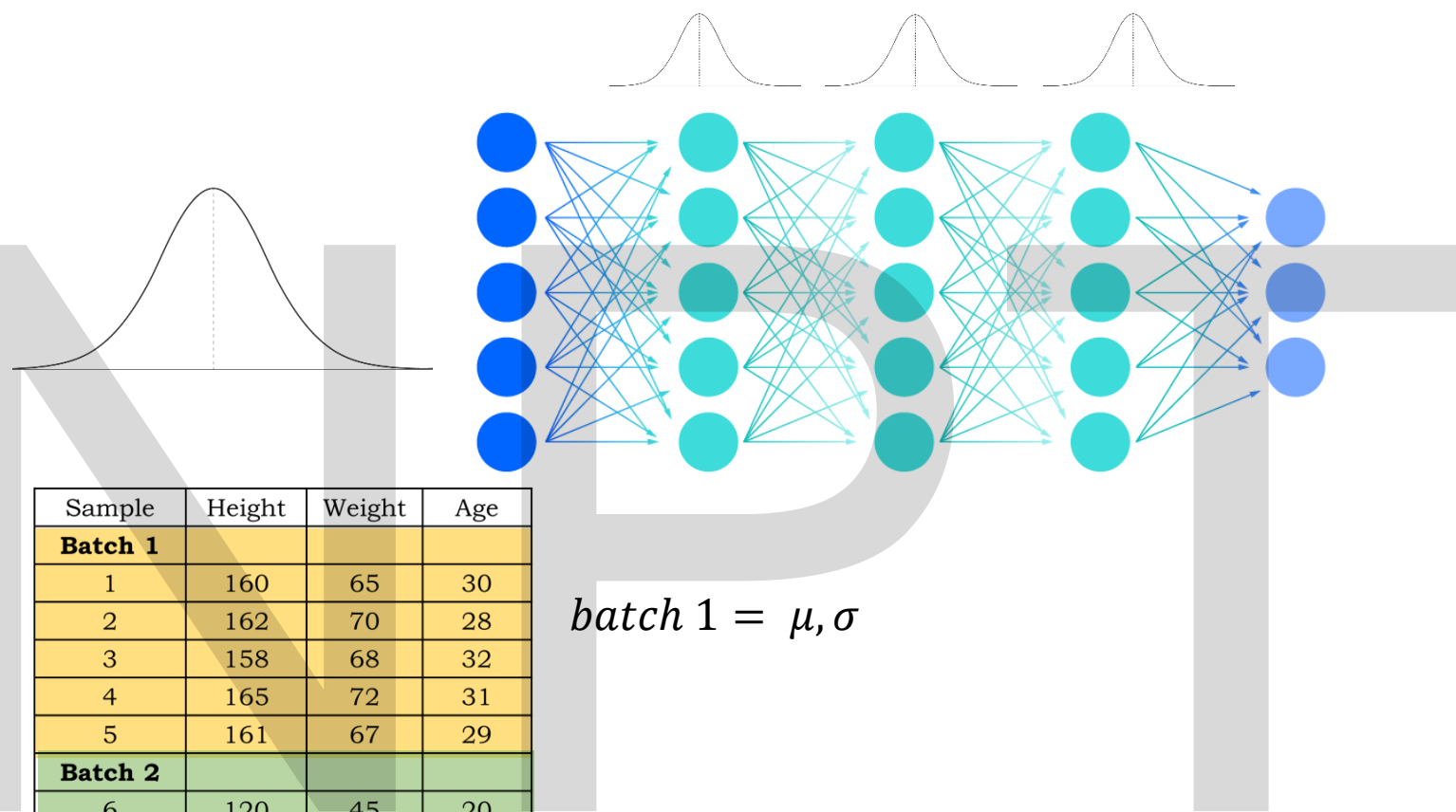
```
from keras.layers import Dense
```

```
model = keras.Sequential([  
    layers.Input(shape=(input_dim,)),  
    layers.Dense(units=128),  
    layers.BatchNormalization(),  
    layers.ReLU(),  
    layers.Dense(units=64),  
    layers.BatchNormalization(),  
    layers.ReLU(),  
    layers.Dense(units=output_dim, activation='softmax')  
])
```

Define the model

```
model = Sequential([  
    Dense(128, activation='relu', input_shape=(input_dim,)),  
    Dense(64, activation='relu'),  
    Dense(num_classes, activation='softmax')  
])
```

Batch Normalization



Sample	Height	Weight	Age
Batch 1			
1	160	65	30
2	162	70	28
3	158	68	32
4	165	72	31
5	161	67	29
Batch 2			
6	120	45	20
7	125	48	22
8	122	43	19
9	130	50	21
10	128	47	23
Batch 3			
11	175	90	40
12	180	85	42
13	178	92	41
14	182	88	43
15	176	89	39

$batch\ 1 = \mu, \sigma$

$batch\ 2 = \mu, \sigma$

$batch\ 3 = \mu, \sigma$

Step 1: Mean and Std Dev

$$\mu_i = \frac{1}{M} \sum Z_i$$

$$\sigma_i = \sqrt{\frac{1}{M} \sum (Z_i - \mu)^2}$$

Step 2: Normalize

$$\hat{Z}_i = \frac{Z_i - \mu_i}{\sigma_i + \epsilon}$$

Step 3: Scale and Shift

$$y = \gamma \odot \hat{Z}_i + \beta$$

Step 4: $a = f(y)$

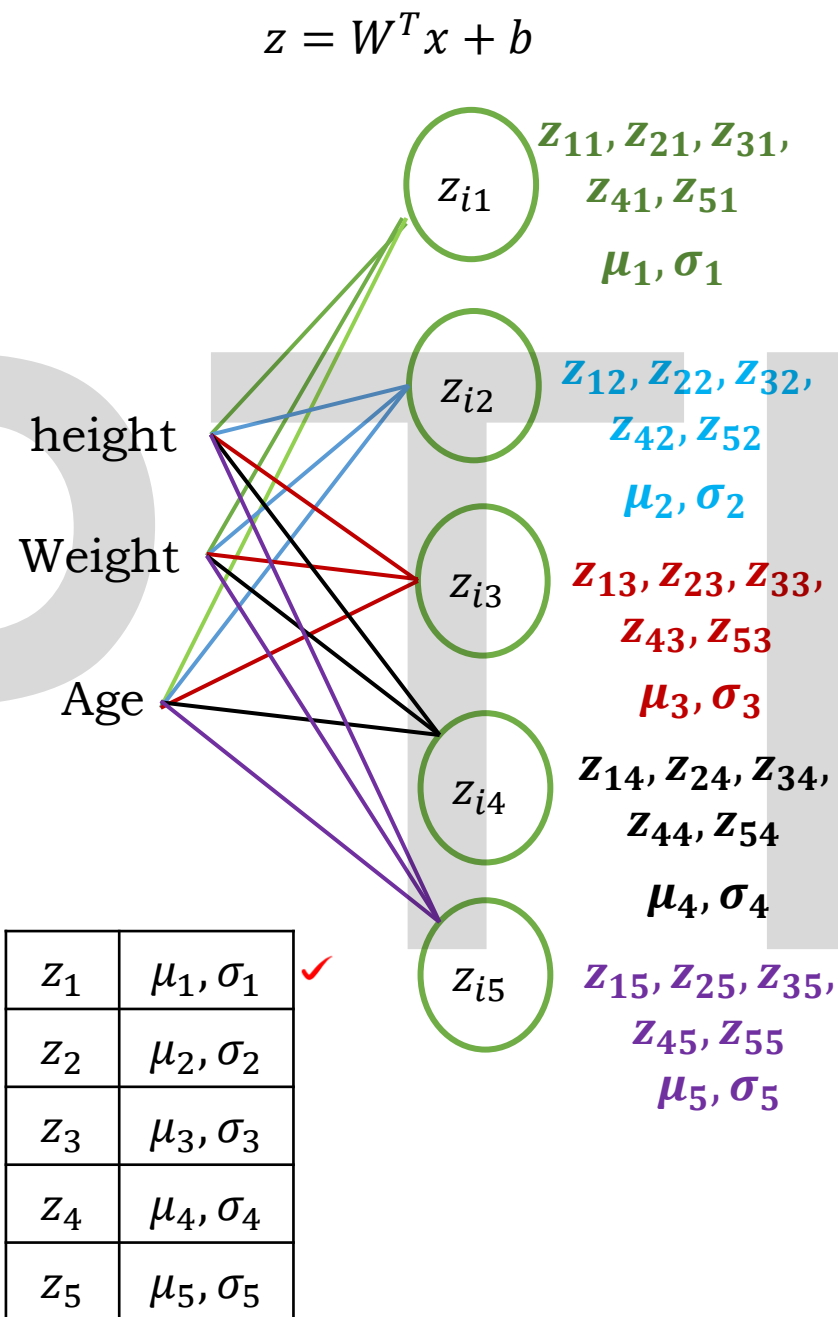
Batch	$\mu(\text{Height})$	$\sigma(\text{Height})$	$\mu(\text{Weight})$	$\sigma(\text{Weight})$	$\mu(\text{Age})$	$\sigma(\text{Age})$
1	170.0	1.41	70.0	1.41	25.4	1.02
2	125.0	3.69	46.6	2.42	21.0	1.41
3	178.2	2.40	88.8	2.42	41.0	1.41

Sample	Height	Weight	Age
Batch 1			
1	160	65	30
2	162	70	28
3	158	68	32
4	165	72	31
5	161	67	29

	z_1	z_2	z_3	z_4	z_5
S_1	z_{11}	z_{12}	z_{13}	z_{14}	z_{15}
S_2	z_{21}	z_{22}	z_{23}	z_{24}	z_{25}
S_3	z_{31}	z_{32}	z_{33}	z_{34}	z_{35}
S_4	z_{41}	z_{42}	z_{43}	z_{44}	z_{45}
S_5	z_{51}	z_{52}	z_{53}	z_{54}	z_{55}

$$\mu_1 = \frac{z_{11} + z_{21} + z_{31} + z_{41} + z_{51}}{5}$$

$$\sigma_1^2 = \frac{(z_{11} - \mu_1)^2 + \dots + (z_{51} - \mu_1)^2}{5}$$



Step 1: Mean and Std Dev

$$\mu_i = \frac{1}{M} \sum Z_i$$

$$\sigma_i = \sqrt{\frac{1}{M} \sum (Z_i - \mu)^2}$$

Step 2: Normalize

$$\hat{Z}_i = \frac{Z_i - \mu_i}{\sigma_i + \varepsilon}$$

Step 3: Scale and Shift

$$y = \gamma \odot \hat{Z}_i + \beta$$

Step 4: $a = f(y)$

$$z = [2,4,6,8,10]$$

z ₁₁	z ₁₂	z ₁₃	z ₁₄	z ₁₅
z ₂₁	z ₂₂	z ₂₃	z ₂₄	z ₂₅
z ₃₁	z ₃₂	z ₃₃	z ₃₄	z ₃₅
z ₄₁	z ₄₂	z ₄₃	z ₄₄	z ₄₅
z ₅₁	z ₅₂	z ₅₃	z ₅₄	z ₅₅

$$\text{Mean: } \mu = \frac{2+4+6+8+10}{5} = 6$$

$$\text{Variance: } \sigma^2 = \frac{(2-6)^2+(4-6)^2+(6-6)^2+(8-6)^2+(10-6)^2}{5} = \frac{40}{5} = 8$$

$$(\text{std dev}) \sigma = \sqrt{8} = 2.828$$

-1.4144	z ₁₂	z ₁₃	z ₁₄	z ₁₅
-0.7072	z ₂₂	z ₂₃	z ₂₄	z ₂₅
0.000	z ₃₂	z ₃₃	z ₃₄	z ₃₅
+0.7072	z ₄₂	z ₄₃	z ₄₄	z ₄₅
+1.4144	z ₅₂	z ₅₃	z ₅₄	z ₅₅

$$\hat{Z}_i = \frac{Z_i - \mu_i}{\sigma_i + \varepsilon}$$

$$\varepsilon = 10^{-5}$$

$$y = \gamma \odot \hat{Z}_i + \beta$$

$\gamma = 1 \ \beta = 0$ (For the 1st mini – batch),

Network produces predictions → compute **loss**.

$$\gamma_{new} = \gamma_{old} - \eta \frac{\partial L}{\partial \gamma}$$

$$\beta_{new} = \beta_{old} - \eta \frac{\partial L}{\partial \beta}$$

$$y = \gamma \odot \hat{Z}_i + \beta$$

$\gamma = 1 \ \beta = 0$

Z _i	$\hat{Z}_i$	y
2	-1.4144	-1.4144
4	-0.7072	-0.7072
6	0.000	0.000
8	+0.7072	+0.7072
10	+1.4144	+1.4144

$$\text{moving_mean} \leftarrow (1 - \text{momentum}) * \mu_i + \text{momentum} * \text{moving_mean}$$

$$\text{moving_variance} \leftarrow (1 - \text{momentum}) * \sigma_i + \text{momentum} * \text{moving_variance}$$

Mini-batch 1

			moving_mean	moving_var
z_1	μ_1	σ_1	$mm_1^1 = (1 - \alpha) * \mu_1$	$m\sigma_1^1 = (1 - \alpha) * \sigma_1 + \alpha$
z_2	μ_2	σ_2	$mm_1^2 = (1 - \alpha) * \mu_2$	$m\sigma_1^2 = (1 - \alpha) * \sigma_2 + \alpha$
z_3	μ_3	σ_3	$mm_1^3 = (1 - \alpha) * \mu_3$	$m\sigma_1^3 = (1 - \alpha) * \sigma_3 + \alpha$
z_4	μ_4	σ_4	$mm_1^4 = (1 - \alpha) * \mu_4$	$m\sigma_1^4 = (1 - \alpha) * \sigma_4 + \alpha$
z_5	μ_5	σ_5	$mm_1^5 = (1 - \alpha) * \mu_5$	$m\sigma_1^5 = (1 - \alpha) * \sigma_5 + \alpha$

**moving_mean = 0 and
moving_var = 1**

Mini-batch 2

			moving_mean	moving_var
z_1	μ_1	σ_1	$mm_2^1 = (1 - \alpha) * \mu_1 + \alpha * mm_1^1$	$m\sigma_2^1 = (1 - \alpha) * \sigma_1 + \alpha * m\sigma_1^1$
z_2	μ_2	σ_2	$mm_2^2 = (1 - \alpha) * \mu_2 + \alpha * mm_1^2$	$m\sigma_2^2 = (1 - \alpha) * \sigma_2 + \alpha * m\sigma_1^2$
z_3	μ_3	σ_3	$mm_2^3 = (1 - \alpha) * \mu_2 + \alpha * mm_1^3$	$m\sigma_2^3 = (1 - \alpha) * \sigma_3 + \alpha * m\sigma_1^3$
z_4	μ_4	σ_4	$mm_2^4 = (1 - \alpha) * \mu_2 + \alpha * mm_1^4$	$m\sigma_2^4 = (1 - \alpha) * \sigma_4 + \alpha * m\sigma_1^4$
z_5	μ_5	σ_5	$mm_2^5 = (1 - \alpha) * \mu_2 + \alpha * mm_1^5$	$m\sigma_2^5 = (1 - \alpha) * \sigma_5 + \alpha * m\sigma_1^5$

$$\text{moving_mean} \leftarrow (1 - \text{momentum}) * \mu_i + \text{momentum} * \text{moving_mean}$$

$$\text{moving_variance} \leftarrow (1 - \text{momentum}) * \sigma_i + \text{momentum} * \text{moving_variance}$$

Mini-batch 2

			moving_mean	moving_var
z_1	μ_1	σ_1	$mm_2^1 = (1 - \alpha) * \mu_1 + \alpha * mm_1^1$	$m\sigma_2^1 = (1 - \alpha) * \sigma_1 + \alpha * m\sigma_1^1$
z_2	μ_2	σ_2	$mm_2^2 = (1 - \alpha) * \mu_2 + \alpha * mm_1^2$	$m\sigma_2^2 = (1 - \alpha) * \sigma_2 + \alpha * m\sigma_1^2$
z_3	μ_3	σ_3	$mm_2^3 = (1 - \alpha) * \mu_2 + \alpha * mm_1^3$	$m\sigma_2^3 = (1 - \alpha) * \sigma_3 + \alpha * m\sigma_1^3$
z_4	μ_4	σ_4	$mm_2^4 = (1 - \alpha) * \mu_2 + \alpha * mm_1^4$	$m\sigma_2^4 = (1 - \alpha) * \sigma_4 + \alpha * m\sigma_1^4$
z_5	μ_5	σ_5	$mm_2^5 = (1 - \alpha) * \mu_2 + \alpha * mm_1^5$	$m\sigma_2^5 = (1 - \alpha) * \sigma_5 + \alpha * m\sigma_1^5$

$$\hat{z}_1 = \frac{z_1 - mm_3^1}{m\sigma_3^1 + \varepsilon}$$

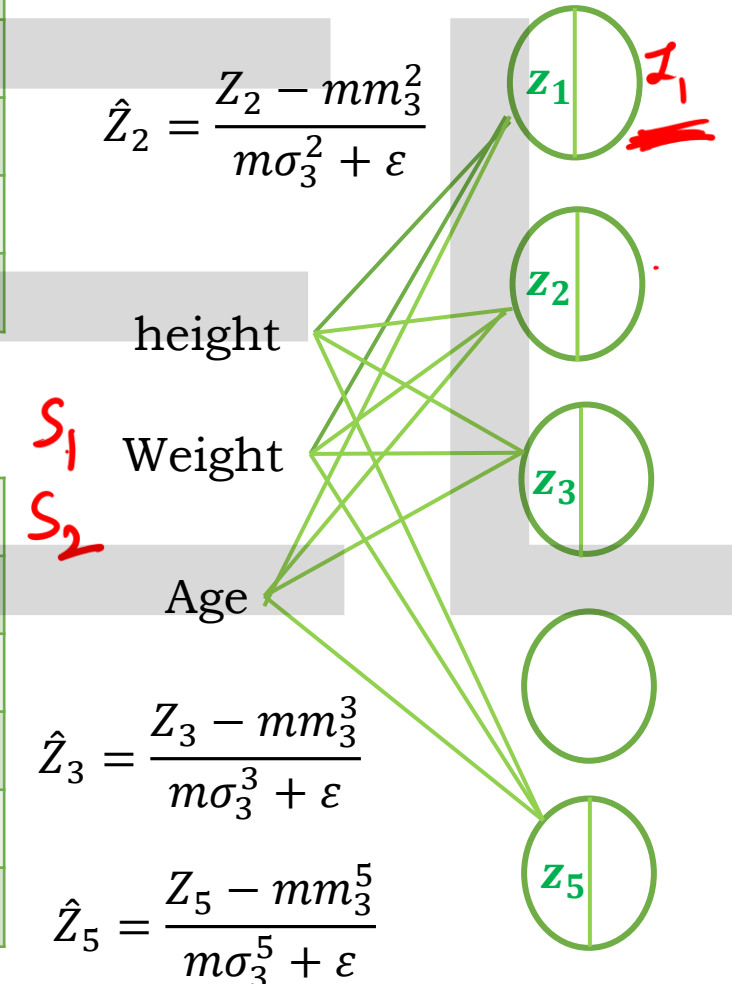
$$\hat{z}_2 = \frac{z_2 - mm_3^2}{m\sigma_3^2 + \varepsilon}$$

Mini-batch 3 ✓

			moving_mean	moving_var
z_1	μ_1	σ_1	$mm_3^1 = (1 - \alpha) * \mu_1 + \alpha * mm_2^1$	$m\sigma_3^1 = (1 - \alpha) * \sigma_1 + \alpha * m\sigma_2^1$
z_2	μ_2	σ_2	$mm_3^2 = (1 - \alpha) * \mu_2 + \alpha * mm_2^2$	$m\sigma_3^2 = (1 - \alpha) * \sigma_2 + \alpha * m\sigma_2^2$
z_3	μ_3	σ_3	$mm_3^3 = (1 - \alpha) * \mu_2 + \alpha * mm_2^3$	$m\sigma_3^3 = (1 - \alpha) * \sigma_3 + \alpha * m\sigma_2^3$
z_4	μ_4	σ_4	$mm_3^4 = (1 - \alpha) * \mu_2 + \alpha * mm_2^4$	$m\sigma_3^4 = (1 - \alpha) * \sigma_4 + \alpha * m\sigma_2^4$
z_5	μ_5	σ_5	$mm_3^5 = (1 - \alpha) * \mu_2 + \alpha * mm_2^5$	$m\sigma_3^5 = (1 - \alpha) * \sigma_5 + \alpha * m\sigma_2^5$

$$\hat{z}_3 = \frac{z_3 - mm_3^3}{m\sigma_3^3 + \varepsilon}$$

$$\hat{z}_5 = \frac{z_5 - mm_3^5}{m\sigma_3^5 + \varepsilon}$$



Batch Norm during Inference

During training

Moving averages:

$$\text{moving_mean} \leftarrow (1 - \text{momentum}) * \mu_i + \text{momentum} * \text{moving_mean}$$

$$\text{moving_variance} \leftarrow (1 - \text{momentum}) * \sigma_i + \text{momentum} * \text{moving_variance}$$

At inference:

Normalize:

$$\hat{Z}_i = \frac{Z_i - \mu_{\text{moving}_i}}{\sigma_{\text{moving}_i}}$$

Scale and Shift:

$$y = \gamma \odot \hat{Z}_i + \beta$$

Step 1: Mean and Std Dev

$$\mu_i = \frac{1}{M} \sum Z_i$$

$$\sigma_i = \sqrt{\frac{1}{M} \sum (Z_i - \mu)^2}$$

Step 2: Normalize

$$\hat{Z}_i = \frac{Z_i - \mu_i}{\sigma_i + \varepsilon}$$

Step 3: Scale and Shift

$$y = \gamma \odot \hat{Z}_i + \beta$$

Step 4: $a = f(y)$

Batch normalization explanation

- There are two learnable parameters called beta and gamma.
- Two non-learnable parameters (Mean Moving Average and Variance Moving Average) are saved as part of the 'state' of the Batch Norm layer.

Batch Normalization Implementation in CNN

```
model = Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.BatchNormalization(), # Batch Normalization after Conv2D
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.BatchNormalization(), # Batch Normalization after another Conv2D
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.BatchNormalization(), # Batch Normalization after Dense layer
    layers.Dense(10, activation='softmax')
])
```

Summary

In this session, you explored how to prevent overfitting and improve the generalization of CNN models using **Batch Normalization**.

Reference for batch normalization:

<https://towardsdatascience.com/batch-norm-explained-visually-how-it-works-and-why-neural-networks-need-it-b18919692739/> for good visualization ✓

Next session

In the upcoming session, you will:

- Implement a **complete CNN pipeline** for image classification and also apply the regularization techniques studied today.

NIPTELL

Thank you